

# Project: "Mobile Game Backend Platform" — AWS Networking, IAM & Terraform Module Design

A self-directed learning project designed to build the exact skills the Peak Games Cloud Platform team uses daily. The project simulates building infrastructure for a mobile game backend, layered in phases that mirror how a real platform team would grow a system.

## Part 1: Learning Outcomes

These outcomes are designed first; the project is built backwards from them.

### AWS Networking Outcomes

By the end of this project, you should be able to:

1. **Design a multi-AZ VPC from scratch** — explain the role of public, private, and isolated subnets, when to use each, and how to size CIDR blocks for future growth.
2. **Explain the full packet path** for a request entering the system: client → Route 53 → ALB → target group → EC2 in private subnet → RDS in isolated subnet. Identify every security group, NACL, and route table the packet touches.
3. **Configure egress correctly** — understand NAT gateways vs NAT instances, when VPC endpoints (Gateway vs Interface) save money and improve security, and the cost implications of each.
4. **Distinguish security groups from NACLs** — stateful vs stateless, where each is enforced, and when to reach for which.
5. **Implement defense in depth** — bastion hosts vs SSM Session Manager, private vs public subnets for compute, and isolated subnets for data.
6. **Debug network failures methodically** — given "my EC2 can't reach the internet," produce an ordered checklist of what to inspect (route table → NAT → SG → NACL → DNS).
7. **Understand cross-AZ data transfer costs** and design to minimize them.
8. **Set up VPC Flow Logs** and read them to identify rejected traffic.

# AWS IAM Outcomes

By the end of this project, you should be able to:

9. **Distinguish principals** — IAM users, roles, groups, service-linked roles, and when each is appropriate.
10. **Write least-privilege policies** — including resource-level permissions, conditions ( `aws:SourceIp` , `aws:PrincipalTag` ), and the difference between identity-based and resource-based policies.
11. **Implement the assume-role pattern** — trust policies, `sts:AssumeRole` , and why this is the foundation of secure cross-account access.
12. **Configure instance profiles** so EC2 instances get credentials without baked-in keys.
13. **Use OIDC federation** to let GitHub Actions deploy to AWS without long-lived credentials.
14. **Read and write IAM policy documents fluently** — understand `Effect` , `Action` , `Resource` , `Condition` , `Principal` , and the policy evaluation logic (explicit deny > explicit allow > implicit deny).
15. **Use IAM Access Analyzer** to find overly permissive policies.
16. **Apply permissions boundaries** to limit what roles can grant other roles.

# Terraform Module Design Outcomes

By the end of this project, you should be able to:

17. **Identify what belongs in a module vs root configuration** — the "shouldn't repeat yourself, but also shouldn't abstract prematurely" judgment call.
18. **Design clean module interfaces** — minimal required variables, sensible defaults, well-typed inputs, useful outputs.
19. **Version modules properly** using Git tags and `?ref=` syntax, and explain why versioning matters.
20. **Compose modules** — when a module should call another module vs when the root should orchestrate.
21. **Use `for_each` and `count`** correctly, including the gotchas (when to use which, why `for_each` is usually safer).
22. **Manage state professionally** — remote backend on S3 with DynamoDB locking, state separation per environment, `terraform_remote_state` for cross-stack references.
23. **Use workspaces vs separate state files** — and articulate why workspaces are often the wrong choice for environment separation.
24. **Write modules that are testable** — sensible outputs, clear variable validation, and basic `terratest` familiarity.

25. **Handle the "drift" problem** — what happens when someone clicks in the console, how `terraform plan` reveals it, and strategies for handling it.
26. **Refactor without destroying resources** — `moved blocks`, `terraform state mv`, `import blocks`.

## Part 2: Project Design

### Premise

You are the sole platform engineer for a fictional mobile game studio called **Pumpkin Games**. The product team is launching a casual puzzle game and needs you to build the backend infrastructure. The infrastructure must be:

- **Reproducible** — staging and production are built from the same code.
- **Secure** — no public databases, no hardcoded credentials, least-privilege IAM everywhere.
- **Observable** — flow logs, CloudTrail, and basic CloudWatch in place.
- **Cost-aware** — you should be able to explain every dollar.

You will build this in **6 phases**, each adding complexity. Each phase has explicit deliverables and reflection questions. Do not skip the reflection questions — they are where the learning compresses.

### Repository Layout (Target State)

By the end of the project, your repository should look like this:

```
pumpkin-platform/  
├── modules/  
│   ├── vpc/  
│   ├── security-baseline/  
│   ├── ec2-app/  
│   ├── alb/  
│   ├── rds-mysql/  
│   └── bastion/  
├── environments/  
│   ├── staging/  
│   │   ├── main.tf  
│   │   ├── backend.tf  
│   │   └── terraform.tfvars  
│   └── production/  
│       ├── main.tf  
│       ├── backend.tf  
│       └── terraform.tfvars  
├── bootstrap/  
│   └── (creates the S3 + DynamoDB for state)  
├── .github/workflows/  
│   └── terraform-plan.yml  
└── README.md
```

You will not start with this structure. You will arrive at it. Resist the urge to scaffold everything upfront — the whole point is to feel the pain that makes good structure obvious.

## Phase 1: Single-File Ugly Terraform (Days 1-3)

**Goal:** Build it badly first. Feel the pain.

Write one `main.tf` file (in a single directory, no modules) that creates:

- A VPC with CIDR `10.0.0.0/16`
- Two public subnets in two AZs ( `10.0.1.0/24` , `10.0.2.0/24` )
- An internet gateway
- A route table for public subnets
- One EC2 instance in a public subnet, with a security group allowing SSH from your IP and HTTP from anywhere
- An Elastic IP attached to the instance

- User data that installs nginx and serves a "Hello from Pumpkin Games" page

Use `aws_*` resources directly. No modules. No variables (yes, really — hardcode everything). No remote state.

**Validate it works:** `curl` the EIP and see your page.

**Reflection questions (write answers in `phase1-reflection.md`):**

- What happens if you `terraform destroy` and re-apply? Why is the EIP different?
- Where is your state file? What would happen if you deleted it?
- If your laptop died right now, could a teammate continue this work? What would they need?
- Count the lines. How would this scale to 50 services?

## Phase 2: Variables, Outputs, and Remote State (Days 4-5)

**Goal:** Make the configuration parameterized and team-safe.

1. **Bootstrap remote state.** In a separate `bootstrap/` directory, write Terraform that creates:
  - An S3 bucket for state (with versioning enabled, encryption enabled, public access blocked)
  - A DynamoDB table for state locking (with `LockID` as the hash key)Apply this with **local state**, then commit the state file to your repo with a note: "This is the only state file we commit. It bootstraps everything else."
2. **Refactor Phase 1.** Move it into `environments/staging/`. Add a `backend.tf` pointing to your S3 bucket. Migrate state with `terraform init -migrate-state`.
3. **Parameterize.** Introduce variables: `environment`, `vpc_cidr`, `instance_type`, `aws_region`. Move values into `terraform.tfvars`.
4. **Add outputs.** Expose the EIP, the VPC ID, and the subnet IDs.

**Reflection questions:**

- Why does the bootstrap directory use local state? Why is this acceptable?
- What does the DynamoDB table actually prevent? Try running `terraform apply` in two terminals simultaneously to see locking in action.
- What information is now in your state file that wasn't before? (Hint: look at the raw JSON in S3.)
- Why did we put state in S3 instead of GitHub?

## Phase 3: Network Redesign — Public, Private, Isolated (Days 6-9)

**Goal:** Build a proper production-grade network. This is where most of the AWS networking outcomes are exercised.

Redesign the VPC to have **three tiers** across two AZs:

- **Public subnets** ( 10.0.0.0/24 , 10.0.1.0/24 ): ALB lives here. NAT gateways live here.
- **Private subnets** ( 10.0.10.0/24 , 10.0.11.0/24 ): EC2 application instances live here. No public IPs.
- **Isolated subnets** ( 10.0.20.0/24 , 10.0.21.0/24 ): RDS lives here. No route to internet at all.

Add:

- One NAT gateway per AZ (yes, this is more expensive — explain why we do it anyway)
- Three route tables (public, private-per-AZ, isolated)
- VPC endpoints for S3 (Gateway endpoint — it's free) and Systems Manager (Interface endpoints — these cost money but enable SSM Session Manager)
- VPC Flow Logs going to a CloudWatch Log Group

Move the EC2 instance into a private subnet. You should no longer be able to SSH to it directly. Use **SSM Session Manager** to connect instead — this requires the right IAM role on the instance.

Add an Application Load Balancer in the public subnets, with a target group pointing to your EC2 instance.

### Reflection questions:

- Draw the packet path for an HTTP request from your laptop to the nginx instance. List every AWS resource it touches.
- Why is the NAT gateway in the public subnet, not the private one?
- What's the difference between the Gateway endpoint for S3 and an Interface endpoint for SSM? Why do they cost differently?
- If you remove the SSM Interface endpoint, can you still connect via Session Manager? Why or why not?
- Add a rule to a NACL that blocks all egress on port 443. What happens? What does `terraform plan` say? What do the flow logs show?
- What's in your flow logs? Find one ACCEPT and one REJECT. Explain what each means.

## Phase 4: IAM Deep Dive (Days 10-13)

**Goal:** Build the IAM layer properly. This phase is mostly about *writing policies*, not creating resources.

Create the following, all in Terraform:

1. **An EC2 instance role** with:
  - `AmazonSSMManagedInstanceCore` (managed policy, attached)
  - A custom policy allowing the instance to read a specific S3 bucket (resource-level, not `*`)
  - A custom policy allowing the instance to write logs to one specific CloudWatch Log Group
2. **A "deployer" role** that can be assumed by your GitHub Actions workflow via OIDC. This role should:
  - Trust `token.actions.githubusercontent.com` as a federated principal
  - Have a condition restricting it to a specific GitHub repository and branch ( `main` )
  - Have permissions to create/update only the resources Terraform manages in this project
3. **A "developer" role** that engineers assume for read-only access to the AWS console:
  - Read-only on EC2, RDS, S3, VPC
  - Can start SSM sessions but cannot modify infrastructure
  - Has a permissions boundary that prevents privilege escalation
4. **A "break-glass admin" role** with full admin, but:
  - Requires MFA via a condition ( `aws:MultiFactorAuthPresent` )
  - Logs every action (CloudTrail is already on, but document the policy)
  - Can only be assumed from a specific IP range (your home IP)

Write each policy by hand. **Do not use AI to generate the policy documents** — type them yourself. This is the muscle you need.

### Reflection questions:

- Explain the difference between the EC2 instance role's trust policy and its permissions policy.
- Why does OIDC for GitHub Actions remove the need for long-lived AWS access keys? What attack does this prevent?
- A developer with the "developer" role tries to attach the admin policy to their own role. What stops them?
- Write the JSON policy for: "Allow `s3:GetObject` on `arn:aws:s3:::pumpkin-assets-*` only if the request comes from inside the VPC." (Hint: `aws:SourceVpc` condition.)
- Run IAM Access Analyzer on your account. What does it flag? Fix anything legitimate.

## Phase 5: Modularization (Days 14-18)

**Goal:** Now that the code works and you've felt the pain of one giant file, extract modules. The key skill is knowing **where to draw the lines**.

Extract the following modules, in this order:

1. **modules/vpc/** — Takes a CIDR and region. Outputs subnet IDs (grouped by tier), route tables, NAT gateway IPs. This is the foundational module everything else depends on.
2. **modules/security-baseline/** — Creates VPC Flow Logs, CloudTrail, baseline IAM roles. Takes the VPC ID as input.
3. **modules/alb/** — Creates an ALB with a target group. Takes VPC ID, subnet IDs, certificate ARN (optional). Outputs ALB DNS name and target group ARN.
4. **modules/ec2-app/** — Creates an Auto Scaling Group with launch template, instance profile, and security group. Takes subnet IDs, target group ARN, instance type, AMI ID.
5. **modules/rds-mysql/** — Creates an RDS MySQL instance with a subnet group, parameter group, and security group. Takes isolated subnet IDs, a list of SGs allowed to connect, instance class. Outputs the endpoint.
6. **modules/bastion/** (*optional*) — Skip this if using SSM Session Manager (preferred). Include it only as an exercise in writing a "deprecated" module pattern.

### For each module, you must:

- Write a `README.md` with: purpose, inputs (with types and descriptions), outputs, example usage.
- Use variable `"..." { type = ..., description = ..., default = ... }` for every variable.
- Validate inputs where reasonable (e.g., `validation` block ensuring CIDR is a valid format).
- Use `for_each` (not `count`) where you create multiple of the same resource.
- Tag every resource with `Environment`, `ManagedBy = "terraform"`, `Module = "vpc"` (etc.), and `Project = "pumpkin"`. Use a shared `locals` block or a `tags` variable.

### Then refactor your environments:



```
# environments/staging/main.tf
"vpc" ">module "vpc" {
  source = "../modules/vpc"

  environment = "staging"
  cidr_block   = "10.0.0.0/16"
  azs          = ["eu-central-1a", "eu-central-1b"]
}

"alb" ">module "alb" {
  source = "../modules/alb"

  vpc_id           = module.vpc.vpc_id
  public_subnet_ids = module.vpc.public_subnet_ids
  environment      = "staging"
}

# ... and so on
```

**The hard part:** Use `moved` blocks to refactor existing resources into modules **without destroying and recreating them**. This is the skill that separates juniors from people who can be trusted with production.

### Reflection questions:

- For each module: what would happen if you split it further? What would happen if you merged it with another? Defend your boundaries.
- Why is `for_each` usually safer than `count` ? Construct an example where using `count` causes resources to be destroyed and recreated when they shouldn't be.
- Your `vpc` module currently has 15 variables. Can you reduce that to 5 without losing flexibility? (Hint: derived values, sensible defaults, removing premature configurability.)
- What happens if you bump your module to use a feature only available in a newer Terraform version? How do you signal version requirements?

## Phase 6: Multi-Environment + CI/CD (Days 19-22)

**Goal:** Make this look like a real platform team's repository.

1. **Create a production environment** that uses the same modules with different inputs (bigger instances, multi-AZ RDS, deletion protection enabled).
2. **Decide: workspaces or directories?** Write a 500-word doc in your repo explaining your choice. (Spoiler: directories with separate state files is almost always correct for prod/staging separation. Workspaces are for ephemeral things like PR previews.)
3. **Set up GitHub Actions** with two workflows:
  - `terraform-plan.yml` : runs on every PR, runs `terraform plan` against staging, posts the plan as a PR comment
  - `terraform-apply.yml` : runs on merge to `main` , requires manual approval for productionBoth use OIDC to assume the deployer role you created in Phase 4. **No AWS access keys in GitHub Secrets.**
4. **Version your modules.** Tag your repo as `v1.0.0` . Change your environment references from `source = "../../modules/vpc"` to `source = "git::https://github.com/you/pumpkin-platform.git//modules/vpc?ref=v1.0.0"` . Discuss tradeoffs in your README.
5. **Introduce a deliberate "drift":** Manually change something in the AWS console (e.g., add a tag, change an SG rule). Run `terraform plan` . Document what happens. Then fix it two ways: (a) by reverting the console change, (b) by updating Terraform to match. Discuss when each is appropriate.

### Reflection questions:

- What problems does pinning modules to versions solve? What problems does it create?
- A teammate runs `terraform apply` from their laptop instead of through CI. What's the danger? How do you prevent it?
- Production and staging share the same module code but have different state files. What's the risk if they shared state? What's the risk if they used completely different module code?
- Your CI runs `terraform plan` on every PR. What information leaks in the plan output? (Hint: search the output for the word "sensitive" — and then consider what isn't marked sensitive but should be.)

## Part 3: Final Capstone Challenges

After Phase 6, you should have a working, modular, multi-environment platform. To consolidate, attempt these challenges:

## Challenge A: The 3am Page

Someone deleted the route table for your private subnets. The application is down. Without using `terraform apply`, identify what broke using only the AWS console and your understanding of networking. Then, restore service. Then, write a postmortem: what happened, why your monitoring didn't catch it instantly, what you'd add.

## Challenge B: The New Hire

Pretend a new engineer joins your team tomorrow. Write a `CONTRIBUTING.md` that explains:

- How to set up their environment
- How to make changes safely
- How the module versioning works
- What "good" looks like for a PR

If you can't explain it clearly, you don't understand it yet.

## Challenge C: The Cost Audit

Add up the monthly cost of running your `production` environment 24/7. Identify the three biggest line items. Propose three changes that would reduce cost by at least 30% without compromising reliability. (Hint: NAT gateways, single-AZ RDS for non-critical envs, instance right-sizing.)

## Challenge D: The Security Review

Run AWS IAM Access Analyzer, AWS Trusted Advisor (free tier), and `tfsec` (or `checkov`) against your code. Fix everything that's a real issue. Document anything you choose to suppress, with justification.

## Part 4: How to Use This Project

**Pace:** Roughly 3-4 weeks of evening/weekend work, or 2 weeks full-time. Do not rush. The reflection questions are not optional decoration — they are where understanding actually consolidates.

**Cost:** If you keep things small and tear down at the end of each session, this project will cost roughly 20 — 50 in AWS charges over its duration. The biggest cost driver is NAT gateways (

32/month each if left running). Always `terraform destroy` at end of session, and verify in the console.

### Tooling to install:

- Terraform 1.6+ (or OpenTofu)
- AWS CLI v2
- `tflint` , `tfsec` , `terraform-docs` (these will be used by your CI in Phase 6)

**What to commit publicly:** All of it. A public repo demonstrating this work is portfolio-grade evidence for any platform/DevOps role. Just make sure your `terraform.tfvars` and any state files are gitignored.

**When you finish:** Open it for review. Ask someone (mentor, MAGIBU peer, Reddit's r/devops, or your future Peak Games mentor) to review your module interfaces. Real review is the final compression of learning.

## Mapping: Outcomes → Phases

| Outcome                       | Primary Phase        |
|-------------------------------|----------------------|
| 1 (multi-AZ VPC)              | Phase 3              |
| 2 (packet path)               | Phase 3              |
| 3 (egress, NAT, endpoints)    | Phase 3              |
| 4 (SG vs NACL)                | Phase 3              |
| 5 (defense in depth, SSM)     | Phase 3, 4           |
| 6 (network debugging)         | Phase 3, Challenge A |
| 7 (cross-AZ costs)            | Challenge C          |
| 8 (Flow Logs)                 | Phase 3              |
| 9-16 (IAM)                    | Phase 4              |
| 17 (what belongs in a module) | Phase 5              |
| 18 (module interfaces)        | Phase 5              |

| Outcome                       | Primary Phase |
|-------------------------------|---------------|
| 19 (versioning)               | Phase 6       |
| 20 (composition)              | Phase 5       |
| 21 (for_each vs count)        | Phase 5       |
| 22 (state management)         | Phase 2, 6    |
| 23 (workspaces vs dirs)       | Phase 6       |
| 24 (testable modules)         | Phase 5       |
| 25 (drift)                    | Phase 6       |
| 26 (refactor without destroy) | Phase 5       |

Every outcome maps to an exercise. Every exercise has a reflection. That's the contract.